Multi Robot Planning and Coordination (16-891) Spring 2026 - The Robotics Institute

Homework 1 : Multi-Agent Path Finding (MAPF)

Boxiang Fu

February 5, 2026

Collaborators:

1. F.D.C. Plato: Emotional support cat.

1 Task 1: Implementing Joint-State A* and Space-Time A*

(1 points) Q - In practice, wheeled robots often follow the turn-and-move paradigm, where, at each time step, they can either wait, rotate 90° clockwise, rotate 90° counter-clockwise, or move forward by one cell in the direction they are pointing. Given a fleet of M such robots, what is the maximum branching factor for the search tree of joint-state A*?

A - The maximum branching factor is 4^M, corresponding to 4 possible branches for each robot.

2 Task 2: Implementing Prioritized Planning

(2 points) Q - Document the changes you made in your code to handle goal related edge-cases and the failure condition. Also, state and justify the upper bound you used in your code (i.e., why they can address the issues and why they preserve the optimality and completeness of the A* search).

A - For Case 1, I modified the goal acceptance condition in `a_star` so that a path is only accepted if the goal location is not forbidden by any future vertex constraint. This is done by adding a `goal_is_blocked_in_future` function test. It makes sure A* is terminated only when it no longer needs to move aside in further timesteps for higher priority agents.

For Case 2, I added vertex constraints on the goal location for all lower-priority agents from the arrival time through a fixed time horizon. This fixed horizon upper bound is the addition of the map size and the summation of the path lengths of all higher priority agents (as suggested in the Problem Sheet). This upper bound preserves completeness since the agent could theoretically move to any free space in the map (while also evading other agents) under this horizon bound, so a solution must take less time than the bound. The bound preserves optimality since A* still expands

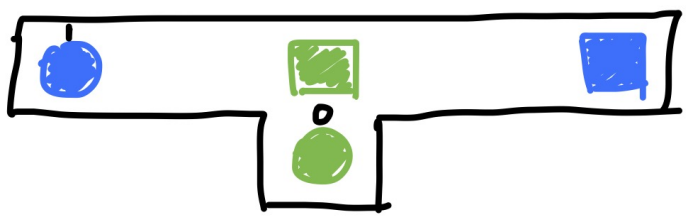
nodes up to the bound. Since an optimal solution must take less time than the bound (as it can travel to any location in the bounded timeframe), setting a bound does not affect optimality.

For Case 3, I enforced a finite time horizon for A* node expansions by disallowing child nodes beyond the horizon inferred from the constraint table (which already includes constraints from Case 2). This prevents agents from "waiting until constraints end" and ensures that truly unsolvable priority orderings correctly return failure. Optimality and completeness of A* is preserved using the same argument as Case 2.

(2 points) Q - Show that Prioritized Planning is Incomplete and Suboptimal

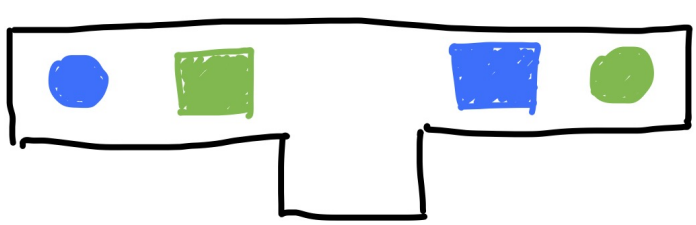
Q - A MAPF instance for which prioritized planning does not find any collision-free solution for a given ordering of the agents, even though an ordering to find a collision-free solution exists.

A - Referring to the Figure below (same case as `exp2_4`) with the green agent having priority. The green agent reaching its goal blocks the corridor, thereby making the lower priority blue agent impossible to reach its goal. If the blue agent had priority, a solution would be found as the green agent could have simply waited.



Q - A MAPF instance for which prioritized planning does not find an collision-free solution, no matter which ordering of the agents it uses, even though a collision-free solution exists.

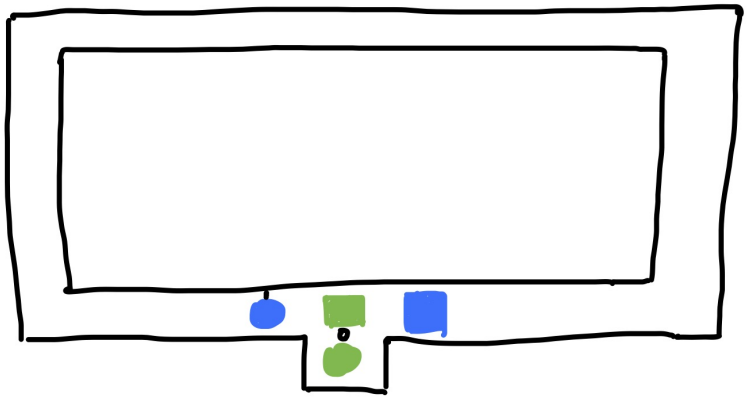
A - Referring to the Figure below, with either agent having priority, a solution does not exist using prioritized planning. Either the blue agent will block the corridor or the green agent will. However, a collision-free solution exists whereby one agent goes into the inlet (passing bay) while the other agent traverses the corridor. However, this cannot be planned using prioritized planning.



Q - A MAPF instance for which prioritized planning finds a suboptimal collision-free solution for a given ordering of the agents, even though an ordering to find an optimal collision-free solution

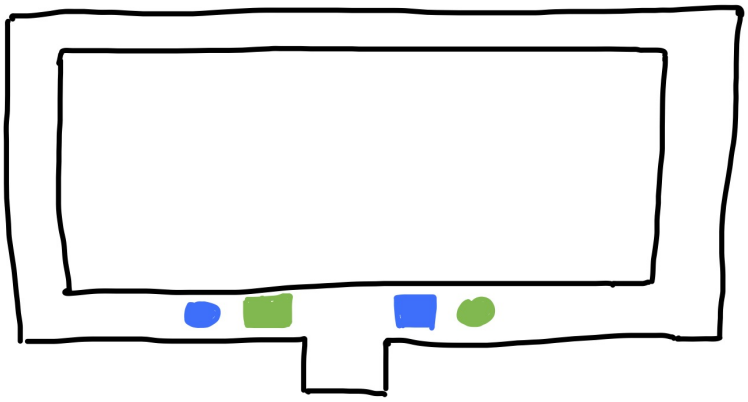
exists.

A - Referring to the Figure below, with the green agent having priority, prioritized planning will lead to a very suboptimal solution. The green agent blocks the corridor, so the blue agent must go the long way to reach its goal. Contrastingly, with the blue agent having priority, the solution will be optimal as the blue agent first plans a path following the short route. The green agent needs to wait one timestep before moving up and blocking the corridor (after the blue agent has passed).



Q - A MAPF instance for which prioritized planning finds a suboptimal collision-free solution, no matter which ordering of the agents it uses, even though an optimal solution exists.

A - Referring to the Figure below, with either agent having priority, prioritized planning will find a very suboptimal solution. Either the blue agent will block the short corridor or the green agent will. This will cause the other agent to go the long way to reach its goal. However, an optimal solution exists where either one of the agent goes into the inlet (passing bay) while the other agent traverses the short corridor. However, this cannot be planned using prioritized planning.



3 Task 3: Conflict-Based Search

(2 points) Q - In the pseudocode for the CBS Algorithm, notice that line 11 doesn't give you much information about which conflict to pick for resolution.

- **Does the strategy of picking a conflict affect the solution cost?**
- **Does the strategy of picking a conflict affect the computation time of the solution?**
- **List three strategies that one could employ to pick conflicts**

A - Picking a conflict does NOT affect solution cost. This is because CBS is an optimal algorithm. The order in which conflicts are resolved may change which nodes are explored first, but it does not change the optimality of the final solution. It does however, affect computation time. Some constraining splits may prune the tree so that the width of the tree is reduced drastically. Others may cause the number of nodes to expand significantly. Some speedup techniques proposed in class include conflict prioritization, symmetry reasoning, and admissible heuristics. Three strategies to pick conflicts could be:

- Cardinality: Pick conflicts in the order of cardinal, semi-cardinal, and last non-cardinal.
- Time: Pick the collision with the minimum timestep.
- Random: Pick any conflict uniformly at random.

4 Task 5: Challenge Your MAPF Solvers

(1 points) Q - Document your observation on their CPU time (seconds) and the sum of costs for each of the solvers on both the maps. Explain the differences you see between them.

A - For `maze_map.txt`:

- Prioritized Planning did not find a solution with default ordering. With the ordering [0, 4, 5, 9, 10, 19, 18, 3, 13, 14, 8, 17, 6, 11, 7, 1, 15, 16], a solution was found in 0.17 seconds in CPU time and had a sum of costs of 821.
- CBS timed out due to A* call limits.
- PBS took 126.28 seconds in CPU time and had a sum of costs of 908.

For `random_map.txt`:

- Prioritized Planning did not find a solution.
- CBS timed out due to A* call limits.
- PBS timed out due to A* call limits.

Prioritized Planning did not find a solution in both cases (for the default sequential ordering) presumably due to the large number of agents in a relatively constrained space. The default priority ordering would cause too many collisions to find a solution. The special ordering [0, 4, 5, 9, 10, 19, 18, 3, 13, 14, 8, 17, 6, 11, 7, 1, 15, 16] found a solution for `maze_map.txt` primarily because it was hand-crafted so that the priority order of path planning did not create collisions. CBS timed out in both cases probably due to the same reason. There were too many collisions between the numerous agents, thereby causing the CBS tree to expand drastically. Speedup methods could be used to refine this algorithm. PBS found a solution for `maze_map.txt` but not `random_map.txt` presumably because there were only 20 agents compared to 80 in the latter. The priority graph for 20 agents was manageable for the algorithm to find a solution in reasonable time, whereas 80 took too long. For `maze_map.txt`, PBS was able to find a solution whereas CBS or Prioritized Planning was not. This could be primarily attributed to the relaxation of the optimality and completeness of the algorithm. In other words, the depth of the priority graph is bounded by $\frac{m(m-1)}{2}$ (so the number of nodes is reasonable), whereas no bounds exist for CBS.